SIMATS ENGINEERING
ASSESSMENT TOOL 2

Course Name	INTERNET PROGRAMMING
Course Code	CSA43
Name	B KIRTHANAA
Register Number	192324244
Course Outcome Covered	CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)
Assessment Tool	Design Task
Weightage	20%

Question 1: Responsive Navigation Bar Design

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.

(a) Identify key components required in a navigation bar for an e-commerce website. (b) Design a responsive navigation bar using appropriate CSS techniques. (c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu). (d) Evaluate the usability and suggest improvements for better user experience.

SOLUTION:

(a) Identify key components required in a navigation bar for an e-commerce website

An e-commerce navigation bar must help shoppers find products and reach their cart quickly. The key components are:

- **Brand logo:** identifies the store and links back to the home page.
- **Primary menu links:** Home, Categories, Deals/Offers and Account for the main sections of the site.
- **Search bar:** lets users find a product directly, which is the most used feature of a shop.
- **Shopping cart icon with item count:** shows the current cart status and leads to checkout.
- **User account / login link:** gives access to orders, wishlist and profile.
- **Hamburger toggle (small screens):** collapses the menu to save space on phones.

(b) Design a responsive navigation bar using appropriate CSS techniques

The design is mobile-first. Flexbox arranges the logo, menu, search box and cart, flex-wrap lets the items move to a new line on small screens, and a media query at 768px switches the menu from a stacked list to a horizontal row. The hamburger menu uses a hidden checkbox and the :checked selector, so it needs no JavaScript.

HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Navigation Bar</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: Arial, sans-serif; }
    .navbar { display: flex; flex-wrap: wrap; align-items: center;
```

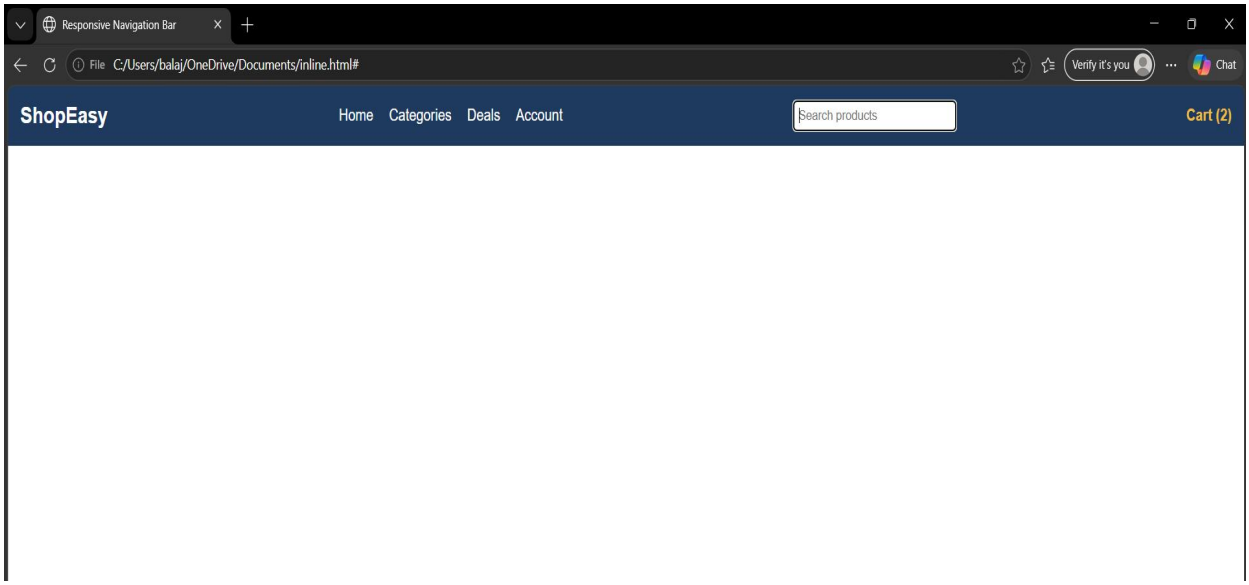
```
            justify-content: space-between; background: #1f3a5f;
            color: #fff; padding: 0.6rem 1rem; }
    .logo { font-size: 1.4rem; font-weight: bold; }
    .menu { list-style: none; display: none; width: 100%;
            flex-direction: column; }
    .menu a { display: block; padding: 0.7rem 0; color: #fff;
            text-decoration: none; }
    .menu a:hover { color: #f4b942; }
    .search { width: 100%; margin: 0.5rem 0; padding: 0.4rem;
            border: 0; border-radius: 4px; }
    .cart { color: #f4b942; font-weight: bold; }
    #toggle { display: none; }
    .burger { font-size: 1.6rem; cursor: pointer; }
    #toggle:checked ~ .menu { display: flex; }

    @media (min-width: 768px) {         /* tablet and desktop */
      .burger { display: none; }
      .menu { display: flex; flex-direction: row; width: auto; gap: 1.2rem;
            order: 2; }
      .search { width: 200px; order: 3; margin: 0; }
      .cart { order: 4; }
    }
  </style>
</head>
<body>
  <nav class="navbar">
    <div class="logo">ShopEasy</div>
    <label for="toggle" class="burger" aria-label="Open menu">&#9776;</label>
    <input type="checkbox" id="toggle">
    <ul class="menu">
      <li><a href="#">Home</a></li>
      <li><a href="#">Categories</a></li>
      <li><a href="#">Deals</a></li>
      <li><a href="#">Account</a></li>
    </ul>
    <input type="search" class="search" placeholder="Search products">
    <div class="cart">Cart (2)</div>
  </nav>
</body>
</html>
```

(c) Demonstrate how the navigation adapts to smaller screens

- **Desktop and tablet (768px and above):** the menu items are shown in one horizontal row with the search box and cart beside them, and the hamburger icon is hidden.
- **Mobile (below 768px):** the menu is hidden (display: none) and only the logo, hamburger icon and cart are visible. Tapping the icon checks the hidden checkbox, and the rule #toggle:checked ~ .menu changes the menu to display: flex, so the links appear stacked.

OUTPUT:



(d) Evaluate the usability and suggest improvements

Evaluation: the bar keeps the most important items (logo, search and cart) visible on every device, the links are large enough to tap, and the layout works without JavaScript. The hover colour on links gives clear feedback.

Suggested improvements:

- Make the bar sticky (position: sticky; top: 0) so the cart and search stay reachable while scrolling.
- Add dropdown or mega-menus for product categories and a search auto-suggest.
- Add ARIA attributes (aria-label, aria-expanded) and visible focus styles for keyboard and screen-reader users.
- Keep touch targets at least 44px high and maintain good colour contrast.

Question 2: Product Hover Effect

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.

(a) Identify suitable CSS properties for hover effects. (b) Suggest appropriate CSS animation or transition for product interaction. (c) Design a hover effect for a product card. (d) Evaluate performance and user experience considerations.

SOLUTION:

(a) Identify suitable CSS properties for hover effects

- **transform:** translateY(), scale() or rotate() lift or enlarge the card.
- **box-shadow:** a deeper shadow gives a feeling of depth.
- **opacity:** fades elements such as the "Add to Cart" button in and out.
- **filter:** brightness() or grayscale() changes the product image.
- **background-color / color / border-color:** give colour feedback on buttons and links.
- **cursor:** pointer shows that the item is clickable.

(b) Suggest appropriate CSS animation or transition for product interaction

A CSS transition is the best choice for hover because it simply animates between two states (normal and hover) and reverses smoothly when the mouse leaves. @keyframes animations are better for effects that run automatically or repeat, such as a pulsing "New" badge. The transition shorthand sets the property, duration and timing function, for example transition: transform 0.3s ease. A duration of 0.2s to 0.4s feels responsive without being distracting.

(c) Design a hover effect for a product card

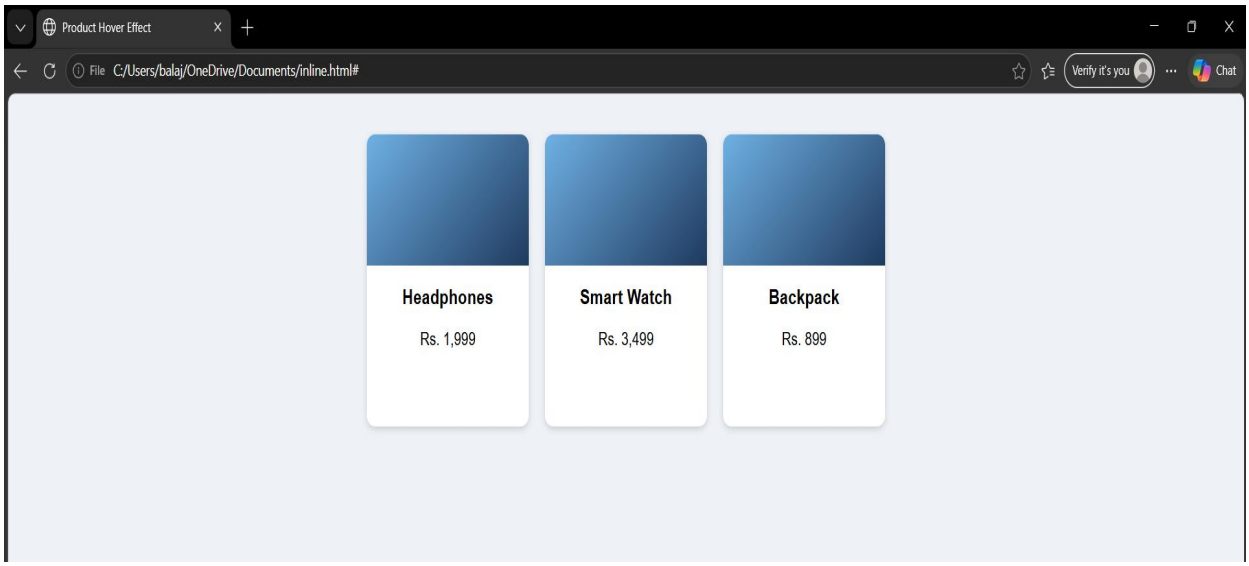
On hover (or keyboard focus) the card lifts up and grows slightly, the shadow becomes deeper, the image brightens and the hidden "Add to Cart" button fades in and slides up.

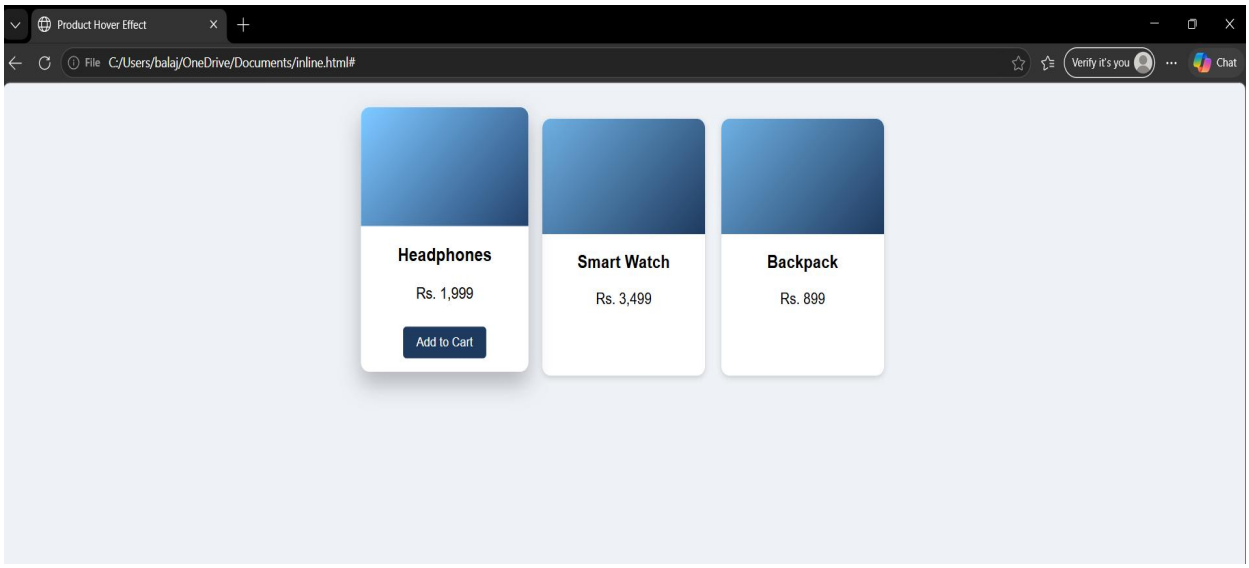
HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product Hover Effect</title>
  <style>
    body { font-family: Arial, sans-serif; display: flex; gap: 20px;
          justify-content: center; padding: 30px; background: #eef2f7; }
    .card { width: 200px; background: #fff; border-radius: 10px;
            overflow: hidden; text-align: center; padding-bottom: 14px;
            box-shadow: 0 2px 6px rgba(0,0,0,.15);
            transition: transform 0.3s ease, box-shadow 0.3s ease; }
    .card:hover, .card:focus-within { transform: translateY(-8px) scale(1.03);
            box-shadow: 0 12px 24px rgba(0,0,0,.25); }
    .img { height: 120px; background: linear-gradient(135deg,#6fb1e4,#1f3a5f);
          transition: filter 0.3s ease; }
    .card:hover .img { filter: brightness(1.15); }
    .btn { margin-top: 8px; padding: 8px 16px; border: 0; border-radius: 4px;
```

```
        background: #1f3a5f; color: #fff; opacity: 0;
        transform: translateY(10px);
        transition: opacity 0.3s ease, transform 0.3s ease; }
   .card:hover .btn, .card:focus-within .btn { opacity: 1; transform: none; }
   @media (prefers-reduced-motion: reduce) {
     .card, .img, .btn { transition: none; }
   }
  </style>
</head>
<body>
  <div class="card"><div class="img"></div><h3>Headphones</h3><p>Rs. 1,999</p><button class="btn">Add to Cart</button></div>
  <div class="card" id="hov"><div class="img"></div><h3>Smart Watch</h3><p>Rs. 3,499</p><button class="btn">Add to Cart</button></div>
  <div class="card"><div class="img"></div><h3>Backpack</h3><p>Rs. 899</p><button class="btn">Add to Cart</button></div>
</body>
</html>
```

OUTPUT:





(d) Evaluate performance and user experience considerations

- **Performance:** transform and opacity are handled by the GPU compositor and do not force the browser to recalculate layout, so the animation stays smooth. Animating width, height, margin or top should be avoided because it triggers layout reflow.
- **Touch devices:** hover does not exist on phones, so the button should not depend only on hover. The :focus-within rule helps keyboard users, and a button that is always visible on small screens is recommended.
- **Accessibility:** the prefers-reduced-motion media query turns animations off for users who are sensitive to motion.

- **User experience:** short, subtle effects give clear feedback; too many or long animations slow down browsing. The will-change property should be used sparingly.

Question 3: Layout Selection (Flexbox vs Grid)

Suppose that a dashboard layout needs to be designed for a web application.

(a) Identify layout requirements of a dashboard (rows, columns, alignment). (b) Compare Flexbox and Grid based on layout needs. (c) Choose the most suitable layout technique and justify your choice. (d) Evaluate the scalability and responsiveness of the chosen approach.

SOLUTION:

(a) Identify layout requirements of a dashboard

- **Rows:** a header row at the top, then rows for summary statistics, charts and tables.
- **Columns:** a fixed-width sidebar for navigation and a flexible main area divided into two or more columns.
- **Spanning:** some blocks (header, statistics) must span several columns, and the sidebar must span several rows.
- **Alignment:** cards must line up in both directions with equal gaps, and content inside each card must be aligned (for example, title left and user name right).
- **Responsiveness:** on small screens the sidebar and cards must stack into a single column.

(b) Compare Flexbox and Grid based on layout needs

Feature	Flexbox	CSS Grid
Dimension	One-dimensional (a row or a column at a time)	Two-dimensional (rows and columns together)
Approach	Content-first: items decide how much space they need	Layout-first: the grid is defined and items are placed in it
Spanning	Cannot span rows and columns together easily	grid-area, grid-column and grid-row span cells easily
Alignment	Excellent for aligning items along one axis	Aligns items in both axes at once
Best for	Navigation bars, toolbars, button groups, card content	Full page layouts, dashboards, galleries
Responsive control	flex-wrap and flex-basis	grid-template-areas, repeat(auto-fit, minmax())

(c) Choose the most suitable layout technique and justify your choice

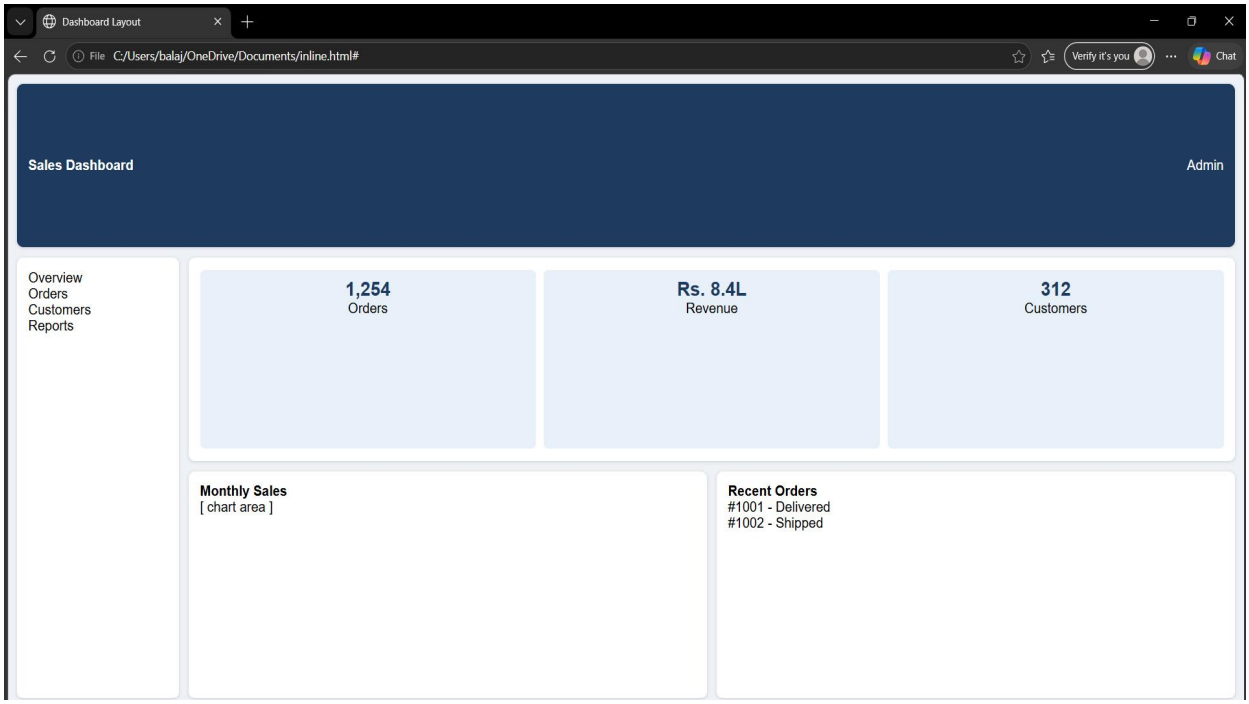
CSS Grid is the most suitable technique for the main dashboard structure, because a dashboard is a two-dimensional layout in which the sidebar spans several rows and the header spans all columns. grid-template-areas describes the whole page in a readable way, and the layout can be rearranged for mobile just by changing the template. Flexbox is still used inside the individual cards (the header bar and the statistics row), where items are arranged along a single axis. Using Grid for the page and Flexbox for the components gives the best result.

HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dashboard Layout</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: Arial, sans-serif; background: #eef2f7; }
    .dashboard { display: grid; gap: 12px; padding: 12px; min-height: 100vh;
      grid-template-columns: 1fr;
      grid-template-areas: "header" "sidebar" "stats" "chart" "table"; }
    .box { background: #fff; border-radius: 8px; padding: 14px;
          box-shadow: 0 1px 4px rgba(0,0,0,.15); }
    .header  { grid-area: header; background: #1f3a5f; color: #fff;
              display: flex; justify-content: space-between;
              align-items: center; }
    .sidebar { grid-area: sidebar; }
    .stats   { grid-area: stats; display: flex; flex-wrap: wrap; gap: 10px; }
```

```
    .stat    { flex: 1 1 120px; text-align: center; background: #e8f0fa;
               border-radius: 6px; padding: 10px; }
    .stat b  { display: block; font-size: 1.4rem; color: #1f3a5f; }
    .chart   { grid-area: chart; min-height: 120px; }
    .table   { grid-area: table; }
    @media (min-width: 900px) {
      .dashboard { grid-template-columns: 200px 1fr 1fr;
        grid-template-areas: "header header header"
                             "sidebar stats stats"
                             "sidebar chart table"; }
    }
  </style>
</head>
<body>
  <div class="dashboard">
    <header class="box header"><strong>Sales Dashboard</strong><span>Admin</span></header>
    <aside class="box sidebar"><p>Overview</p><p>Orders</p><p>Customers</p><p>Reports</p></aside>
    <section class="box stats">
      <div class="stat"><b>1,254</b>Orders</div>
      <div class="stat"><b>Rs. 8.4L</b>Revenue</div>
      <div class="stat"><b>312</b>Customers</div>
    </section>
    <section class="box chart"><strong>Monthly Sales</strong><p>[ chart area ]</p></section>
    <section class="box table"><strong>Recent Orders</strong><p>#1001 - Delivered</p><p>#1002 - Shipped</p></section>
  </div>
</body>
</html>
```

OUTPUT:



(d) Evaluate the scalability and responsiveness of the chosen approach

- **Scalability:** a new widget is added by writing one more grid-area name; existing blocks do not need to be touched.
- **Responsiveness:** the base layout is a single column for phones and the media query at 900px creates the three-column desktop layout, as the output shows.
- **Maintainability:** the named areas make the structure easy to read, and the gap property keeps the spacing consistent.
- **Improvements:** use repeat(auto-fit, minmax(250px, 1fr)) for widget grids, add a collapsible sidebar, and use CSS variables for colours and spacing.

Question 4: Mobile-First Blog Layout

Suppose that a blog page must be designed following a mobile-first approach.

(a) Identify key sections of a blog layout. (b) Design a mobile-first layout using CSS. (c) Demonstrate how the layout scales for larger screens using media queries. (d) Evaluate accessibility and readability of the design.

SOLUTION:

(a) Identify key sections of a blog layout

- **Header:** blog title or logo and a short navigation menu.
- **Main content area:** the list of posts, each in an <article> with a title, date, author and short text.
- **Sidebar:** about the author, categories, recent posts or a search box.
- **Footer:** copyright, contact and social links.

(b) Design a mobile-first layout using CSS

Mobile-first means the basic CSS is written for small screens: one column, full-width cards and comfortable text size. Larger layouts are added afterwards with min-width media queries, so phones download and process only the simple rules. Semantic elements (header, nav, main, article, aside, footer) give the page a clear structure.

HTML & CSS CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mobile-First Blog</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: Georgia, serif; font-size: 1rem; line-height: 1.7;
           color: #222; background: #f5f5f5; }
    /* Mobile-first base styles: single column */
    header { background: #1f3a5f; color: #fff; padding: 1rem; text-align: center; }
    nav a { color: #f4b942; margin: 0 0.5rem; }
    .layout { display: block; padding: 1rem; }
    article { background: #fff; padding: 1rem; margin-bottom: 1rem;
              border-radius: 6px; }
    article h2 { font-size: 1.3rem; margin-bottom: 0.4rem; }
    article .meta { font-size: 0.85rem; color: #555; margin-bottom: 0.5rem; }
    aside { background: #fff; padding: 1rem; border-radius: 6px; }
    footer { background: #1f3a5f; color: #fff; text-align: center; padding: 1rem; }
    a:focus { outline: 3px solid #f4b942; }

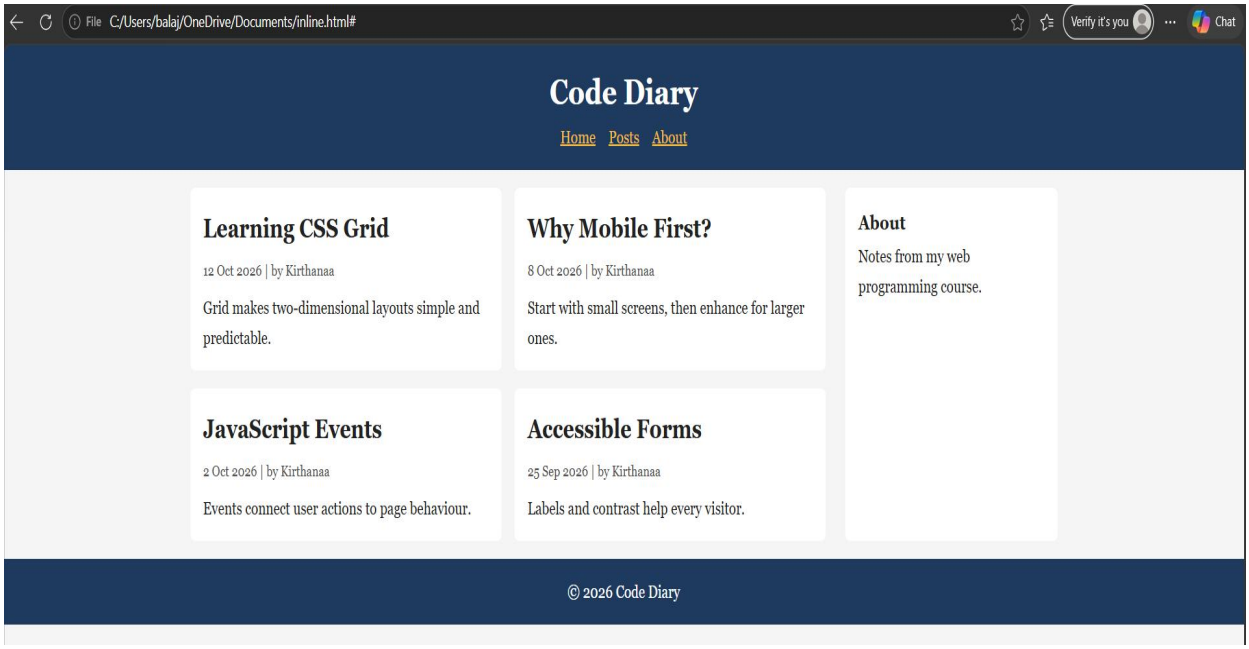
    /* Tablet and above */
    @media (min-width: 768px) {
      .posts { display: grid; grid-template-columns: 1fr 1fr; gap: 1rem; }
      article { margin-bottom: 0; }
    }
    /* Desktop */
    @media (min-width: 1024px) {
      .layout { display: grid; grid-template-columns: 3fr 1fr; gap: 1.5rem;
                max-width: 1100px; margin: 0 auto; }
      article h2 { font-size: 1.5rem; }
    }
  </style>
</head>
<body>
  <header>
    <h1>Code Diary</h1>
    <nav><a href="#">Home</a><a href="#">Posts</a><a href="#">About</a></nav>
  </header>
  <div class="layout">
    <main class="posts">
      <article><h2>Learning CSS Grid</h2><p class="meta">12 Oct 2026 | by Kirthanaa</p><p>Grid makes two-dimensional layouts simple and predictable.</p></article>
      <article><h2>Why Mobile First?</h2><p class="meta">8 Oct 2026 | by Kirthanaa</p><p>Start with small screens, then enhance for larger ones.</p></article>
      <article><h2>JavaScript Events</h2><p class="meta">2 Oct 2026 | by Kirthanaa</p><p>Events connect user actions to page behaviour.</p></article>
      <article><h2>Accessible Forms</h2><p class="meta">25 Sep 2026 | by Kirthanaa</p><p>Labels and contrast help every visitor.</p></article>
```

```
  </main>
  <aside><h3>About</h3><p>Notes from my web programming course.</p></aside>
 </div>
 <footer><p>&copy; 2026 Code Diary</p></footer>
</body>
</html>
```

(c) Demonstrate how the layout scales for larger screens using media queries

- **Mobile (base):** posts and sidebar are stacked in one column.
- **Tablet (min-width: 768px):** the posts are shown in a 2-column grid.
- **Desktop (min-width: 1024px):** the page becomes a 3fr / 1fr grid with the posts on the left and the sidebar on the right, the content is limited to a maximum width of 1100px and the headings become larger.

OUTPUT:



(d) Evaluate accessibility and readability of the design

- **Readability:** a line-height of 1.7, a 1rem base font and short line lengths (because of the max-width) make the text comfortable to read.
- **Accessibility:** semantic HTML helps screen readers, the lang attribute is set, link focus has a visible outline, and the dark text on a white background has strong contrast.
- **Responsive:** the viewport meta tag and relative units (rem, fr) let the text and layout adjust to the device and to the user's font settings.
- **Improvements:** add a skip-to-content link, alt text for all images, a larger tap area for navigation links and a dark mode with prefers-color-scheme.

Question 5: JavaScript Event Handling for Dynamic Menu

Suppose that a website includes a dynamic menu that responds to user interactions.

(a) Identify appropriate JavaScript events for menu interaction. (b) Design event handling logic for menu toggle functionality. (c) Implement interaction for user actions (click/hover). (d) Evaluate performance and suggest improvements.

SOLUTION:

(a) Identify appropriate JavaScript events for menu interaction

- **click:** toggles the menu when the button is pressed or tapped, and selects an item.
- **mouseenter / mouseleave:** opens and closes the menu on hover for mouse users (they do not bubble from child elements, unlike mouseover and mouseout).

- **keydown:** lets keyboard users close the menu with the Escape key.
- **focus / blur:** keep the menu usable with the Tab key.
- **click on document:** detects a click outside the menu so it can be closed.

(b) Design event handling logic for menu toggle functionality

1. Select the button, the menu and a status message with getElementById.
2. Create one function setMenu(open) that adds or removes the CSS class "open", updates aria-expanded and the status text.
3. On button click, flip a "pinned" flag and call setMenu(pinned). Stop propagation so the document click handler does not close the menu immediately.
4. On hover, open the menu only when it is not pinned, and close it on mouseleave.
5. On a document click outside the menu, or on the Escape key, reset the flag and close the menu.
6. On a click on a menu item, show the selected item and close the menu.

(c) Implement interaction for user actions (click/hover)

HTML, CSS & JAVASCRIPT CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Dynamic Menu</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 0; }
    .bar { background: #1f3a5f; color: #fff; padding: 10px 16px;
           display: flex; align-items: center; gap: 16px; position: relative; }
    #menuBtn { background: none; border: 1px solid #fff; color: #fff;
               font-size: 1.2rem; padding: 4px 10px; cursor: pointer; }
    #menu { display: none; position: absolute; top: 100%; left: 16px;
            background: #fff; list-style: none; margin: 0; padding: 0;
            width: 180px; box-shadow: 0 4px 10px rgba(0,0,0,.25); }
    #menu.open { display: block; }
    #menu li a { display: block; padding: 10px 14px; color: #222;
                 text-decoration: none; }
    #menu li a:hover, #menu li a:focus { background: #e8f0fa; }
    #status { padding: 16px; }
  </style>
</head>
<body>
  <div class="bar">
    <button id="menuBtn" aria-expanded="false" aria-controls="menu">&#9776; Menu</button>
    <span>My Website</span>
    <ul id="menu">
      <li><a href="#">Home</a></li>
      <li><a href="#">Products</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
  </div>
  <p id="status">Menu is closed.</p>

  <script>
    const btn = document.getElementById("menuBtn");
    const menu = document.getElementById("menu");
    const status = document.getElementById("status");

    function setMenu(open) {
      menu.classList.toggle("open", open);
      btn.setAttribute("aria-expanded", open);
      status.textContent = open ? "Menu is open." : "Menu is closed.";
    }

    let pinned = false;   // true when the menu was opened by a click

    // Click: pin the menu open, or close it if it is already pinned
    btn.addEventListener("click", function (e) {
      e.stopPropagation();
      pinned = !pinned;
      setMenu(pinned);
    });

    // Hover (mouse devices): preview the menu unless it is pinned open
    const bar = document.querySelector(".bar");
```

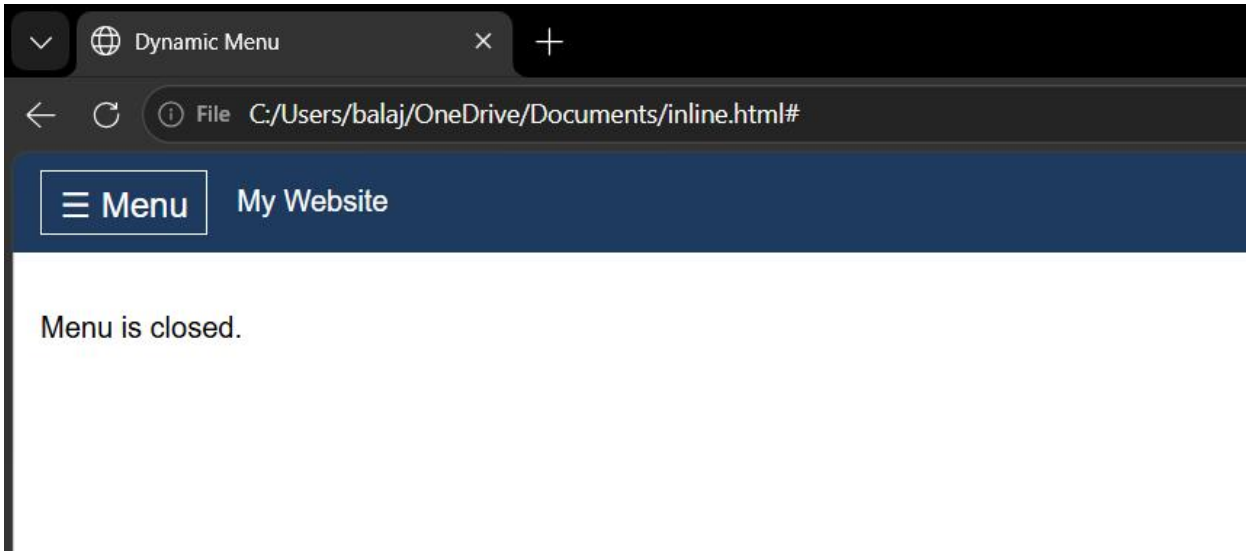
```
    if (window.matchMedia("(hover: hover)").matches) {
      bar.addEventListener("mouseenter", function () { if (!pinned) setMenu(true); });
      bar.addEventListener("mouseleave", function () {
        if (!pinned && menu.classList.contains("open")) setMenu(false);
      });
    }

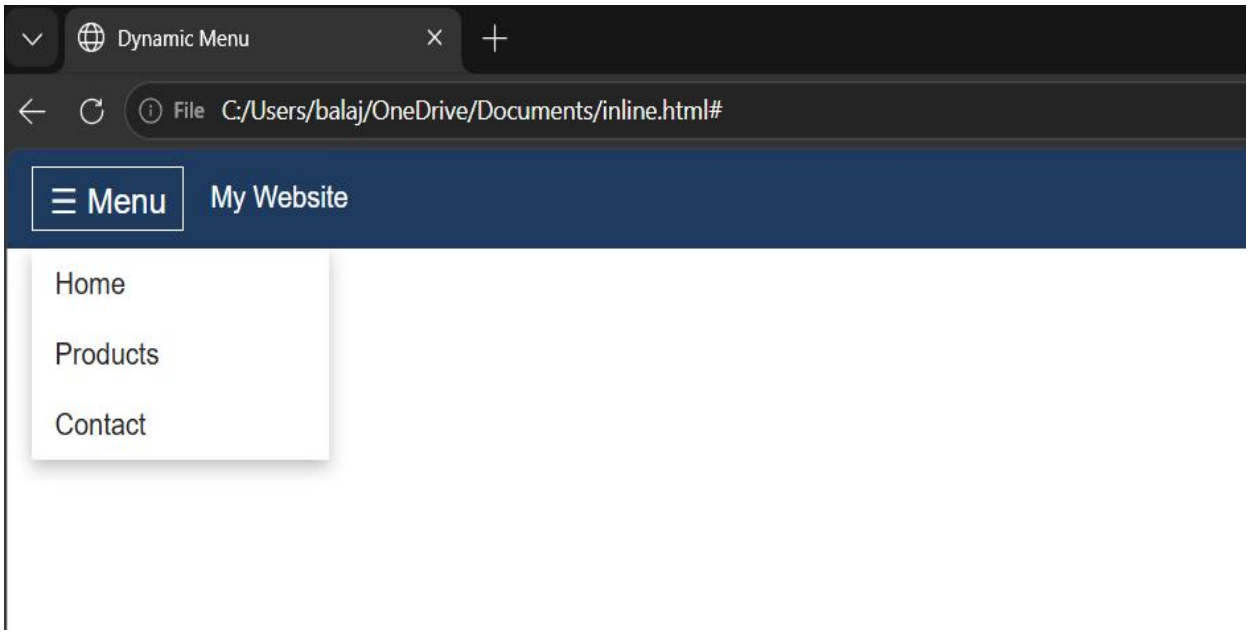
    // Click outside the menu closes it
    document.addEventListener("click", function (e) {
      if (!menu.contains(e.target)) { pinned = false; setMenu(false); }
    });

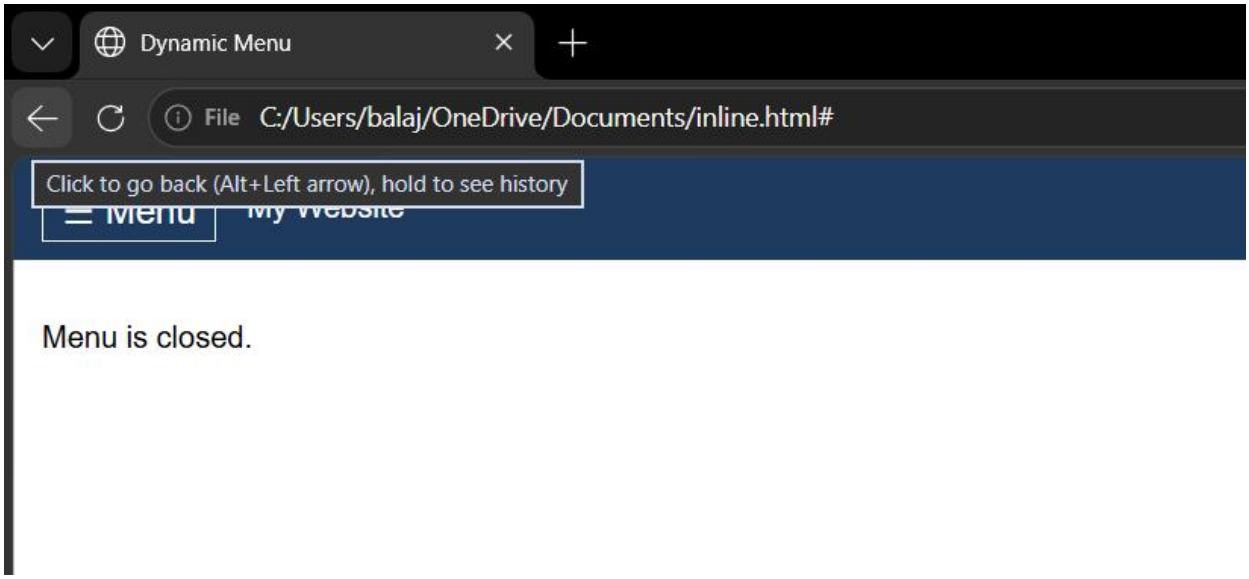
    // Keyboard: Escape closes the menu and returns focus to the button
    document.addEventListener("keydown", function (e) {
      if (e.key === "Escape") { pinned = false; setMenu(false); btn.focus(); }
    });

    // Clicking a menu item shows the choice and closes the menu
    menu.addEventListener("click", function (e) {
      if (e.target.tagName === "A") {
        e.preventDefault();
        status.textContent = "You selected: " + e.target.textContent;
        pinned = false;
        menu.classList.remove("open");
        btn.setAttribute("aria-expanded", false);
      }
    });
  </script>
</body>
</html>
```

OUTPUT:







(d) Evaluate performance and suggest improvements

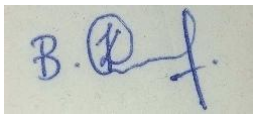
Evaluation: the menu is shown and hidden by toggling a single CSS class, which is fast because no styles are written one by one. The code works with mouse, touch and keyboard, and the aria-expanded attribute tells screen readers whether the menu is open.

Suggested improvements:

- **Event delegation:** one listener on the menu handles clicks for all items instead of one listener per item.
- **Passive listeners and throttling:** use { passive: true } for scroll or touch handlers and throttle or debounce events that fire often (resize, scroll, mousemove).
- **Accessibility:** add arrow-key navigation inside the menu, trap focus while it is open and use role="menu" where appropriate.
- **Animation:** animate the opening with CSS transitions (opacity and transform) instead of JavaScript timers.
- **Clean-up:** remove listeners that are no longer needed and keep the logic in one reusable function or module.

Code of Conduct:

I **B KIRTHANAA 192324244** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student: **B KIRTHANAA**

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation